

Lamina

Source-aware production methods for document workflows

Lamina Project

22 September 2026 — technical design and implementation status

Abstract

Producing a guide, brief or assessment from changing sources requires decisions about authority, context, division of work and repair. Lamina is open-source infrastructure for those decisions. Its current paths include a fixed guide builder, a separate method runtime and a source-aware production engine connected to a local app, CLI, exports and recovery. Source roles, assessment checks and optional PCM rendering cover specific output boundaries. This paper defines source ownership, context selection, dependency and resource edges, cache identity and repair in a task-shaped architecture. Synthetic scheduling and deterministic fixtures establish mechanics. The product hypothesis is that a trace from source decisions to delivered artifacts reduces correction work at matched quality; that outcome still needs a new-source study.

1 The production problem

An operating brief may draw on a current procedure, a superseded note, exceptions in a table and a document used only for style. A single model call can merge old and current rules or lose a condition. Sending every source to every writer costs more and still leaves attention uncertain; long-context retrieval varies by position and task [7]. Parsing and planning can also discard a condition before writing starts.

After a source changes, the owner needs to know which decisions depend on it, what can be reused and whether the corrected file reached its destination. Rebuilding everything costs time and risks new errors; editing final prose alone can leave a false upstream interpretation in place. Lamina treats a *production method* as the versioned transformation from a defined source set to an artifact, with decisions and repair paths retained.

Parallel LLM function calls have been studied [5]; graph workflows, durable state and interruptions already exist [2, 6]. The question here is whether source decisions and repair traces lower total correction work at matched output quality.

2 Evidence from several output forms

A differential reference needs one owner per retrieval target, useful variants and a findable answer. A spoken lesson needs lists, explanation, purposeful repetition and playable audio. An interactive case needs candidate-only findings, examiner answers and new information after actions or elapsed time. A written case adds a setter, question-specific keys and a blind solver limited to the candidate’s staged

view. A technical brief needs an argument that accounts for conflicting sources. They share source identity and revision machinery, but their representations differ.

Reading, selection and delivery leave different evidence. A quotation identifies its source; review checks whether it supports the claim. A script needs an audio check, a case needs an encounter check, and a PDF needs a rendered-page check.

3 A task contract before a graph

The task contract is

$$\mathcal{T} = (G, A, S, P, O, Q, R), \quad (1)$$

where G is the goal, A the audience or operator, S the accepted sources, P their use policy, O the artifact and delivery surface, Q acceptance criteria and R limits on time, model, money, context and machines. A completed run can be assessed against the contract’s stated sources, output and acceptance criteria.

A current specification may govern an operational rule; an older note may document conflict without current authority. A form exemplar may shape presentation while contributing no factual citation, and held-out questions stay outside generation. Lamina accepts authority, supplement, historical and form-exemplar policies for selected teaching sources. Factual sources enter reading; form exemplars enter bounded form context; held-out assessment sources stay excluded. Source identity, hash, role and locator accompany the decision. Policy changes alter the plan’s meaning even when source bytes do not change. Review still has to judge the model’s use of authority.

A prior question may be a form example while another is withheld for evaluation. A source manifest should distinguish discovery, inventory, reading, factual use, form use and withholding. Report coverage at the stage actually measured.

Representation follows the job: guides need claims, conditions and causal steps; differential references need presentation-to-answer objects and variants; interactive cases need states, requests, findings, actions, consequences and marking opportunities. Figures and tables need modality-aware extraction. Review must then examine the appropriate unit: support for a document claim; answer family, exclusion and qualifier for a retrieval target; or reachable findings and treatment-linked observations in a case. The method should name the checks that block delivery, flag review or open a source-policy decision. High-stakes factual claims require authoritative source review beyond a model verdict.

Models may propose objects and relations such as same target, variant, contrast, prerequisite and conflict. Code owns stable IDs, locators, membership and limits. Ambiguous merges remain open. An object may be written, deferred or deliberately omitted according to the task. Every required field needs a consumer in writing, review or repair. Structural accounting should expose the original region and claimed transformation: one owner per extracted item can coexist with a merge that loses an exception.

4 Source windows and ownership

A reading job j receives a focal core C_j and neighboring halo H_j . It may use the halo to interpret the core, while owning extraction only from C_j . For readable accepted regions $S_1, \dots, S_n$, require

$$\bigcup_j C_j = S_{\text{readable}}, \quad C_j \cap C_k = \emptyset \text{ for } j \neq k. \quad (2)$$

This partition prevents duplicate ownership at boundaries. A region inventory must still identify unreadable figures, broken table order and missed clauses. Small windows reduce call size but can lose distant qualifiers; large ones increase repeated context and competition for attention. Choose core size, halo and bridge review for the source and task. Preserve IDs and short supporting spans, and let reviewers reopen the full source when a local packet is insufficient.

4.1 A PDF reference: choose the answer object before the page

In the historical differential-reference work, the source library mixed course teaching, cards, short-answer keys and review prompts. Readers examined owned page or slide cores with neighboring halo, returned source-linked candidates, and passed them to semantic reconciliation. The editorial unit changed with the use task: one design grouped source-derived lists conservatively, while another built presentation-to-answer sheets with discriminating clues and an answered pivot. A diagnosis may recur across presentations because the learner’s question differs. Family-level reconciliation decided which candidates asked the same question; whole-book routing then chose the study sequence and output destination. Code retained source-member and answer-item identity across merges, while models judged equivalence. The chosen answer object determines what rendering must preserve. Page count describes output scope; speed and learning require separate measurements. The historical builders and their aggregate receipts are described in [the workflow design account](#); private source content remains outside this release.

4.2 An audio lesson: route once, then give writers local evidence

The ROUNDS audio factory faced chapters containing hundreds of cards or ordered course units. Its local readers produced idea bundles from owned windows; one director selected the teaching route and natural episode boundaries. Writers could then work on sections in parallel because each received that route, its exact local evidence, shared cross-region resolutions and selected earlier ideas. The earlier ideas were planned context: a writer could know what another section was assigned to teach before its prose existed. A callback that depends on the actual preceding explanation instead requires a hard-result edge or later seam review. The factory kept remote text calls wide and local speech and transcription narrow: one durable FIFO topic worker admitted jobs, while Kokoro rendering and Whisper delivery checks used a constrained media lane on the same Mac. Parallel section writing therefore reduced only the portion of the critical path made of independent model calls. Delivery still required a rendered file, an audit and a player path.

4.3 A staged case: one setter, bounded keys and information-limited solves

In the historical case factory, parallel readers mapped source regions before one setter fixed the whole case opening, question order, staged reveals, marks and idea ownership. Question-specific key writers then received the fixed stems and exact supporting source IDs. Blind solver calls received only the candidate prefix available at each question, even when those calls ran concurrently. An independent judge saw the assembled case, answers and evidence; a material finding reopened named questions and their solve/judge checks. This graph preserves case coherence through a global setter while allowing independent keys and solves to overlap. The live oral-station work used requestable findings, action-triggered reassessment and examiner-controlled release. Those state transitions belong to the encounter contract.

5 Methods and execution semantics

A method names applicability, exclusions, artifacts, worker roles, passed data, dependencies, resource limits and review policy. A task with unsupported source modality or acceptance criteria should reject the method. Current method files declare bounded JSON graphs run through a configured adapter; selection and revision remain manual.

For jobs V , four relations matter:

- A *hard-result edge* $u \rightarrow v$ makes v wait for and receive u 's result.
- A *shared-plan relation* gives jobs the same versioned editorial map without requiring finished prose from another writer.
- An *evidence-validity relation* marks work to reconsider after a source or decision changes; it does not require regeneration of every descendant.
- A *resource relation* assigns finite-capacity model, parser, renderer or review lanes without adding semantic information.

The generic runtime implements hard-result edges and resource lanes. Production uses local parallel reading, a shared route, parallel writing and review. Some jobs need earlier evidence; others need a finished prior section. Both patterns remain useful.

A short coherent file may suit source order. A shared topic board coordinates overlapping files but makes reconciliation a global cost and possible merge error. Chaining finished sections improves continuity while lengthening the critical path and later contexts. An editor over parallel drafts adds review cost. Choose among them using source topology, output form, conflicts, context budget and deadline. Keep a rejected alternative when it helps explain a later repair.

6 Context assembly as an explicit decision

Let I_v be candidate context items for job v , with protected subset P_v required by the task or a consequential claim. Item i uses b_i tokens in the adapter's tokenizer. With input budget B_v after instructions and output reserve, choose X_v so that

$$P_v \subseteq X_v, \quad \sum_{i \in X_v} b_i \leq B_v. \quad (3)$$

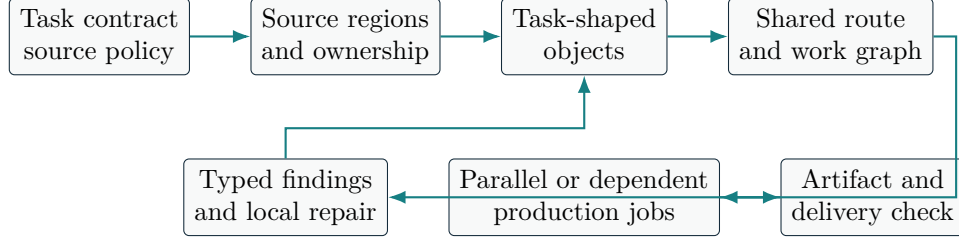


Figure 1: Source-aware production loop. The diagram separates semantic decisions from scheduling and delivery. Review can reopen a specific object or source decision. Full upstream routing remains a design target; current production repairs sections.

Rank optional items for relevance. If P_v alone exceeds B_v , split the job or change scope. Record plan version, included source and object IDs, omitted counts and actual request size. A reviewer needs broader inventory access than a writer to find material lost during selection.

Context amplification is

$$A = \frac{\sum_{v \in V} \text{inputTokens}(v)}{\text{tokens}(\text{distinct accepted source text})} \quad (4)$$

Report A by stage with largest request, source coverage and accepted quality. Repeating a protected rule can prevent error; repeating whole sources can waste cost and attention. The method runner selects top-level task fields and direct dependency results, but does not yet perform the source-aware constrained choice in Eq. 3.

7 Concurrency and measured resource limits

For service times t_v , work $W = \sum_v t_v$ and longest hard-dependency path D , the ideal lower bound with p equivalent workers is

$$T \geq \max \left(D, \frac{W}{p} \right). \quad (5)$$

For lane r with capacity c_r and service work W_r , add its limit:

$$T \geq \max \left(D, \frac{W}{p}, \max_r \frac{W_r}{c_r} \right). \quad (6)$$

These classical work/span and capacity bounds assume compatible time units and a defined task graph [1]. They omit simultaneous resource demands, throttling, setup, queue waits, retries, human review and delivery. A global reconciliation call remains on D regardless of writer count. Waiting for earlier completed prose can reduce duplication while lengthening D ; a shared accelerator may serialize speech rendering and transcription. Batch arrivals and correlated calls can also defeat a steady-state queue model. Measure enqueue, start and end times, lane occupancy and latency tails.

7.1 Historical audio observations at their actual boundaries

An archival analysis of 82 saved ROUNDS V8 rendered *part artifacts* found median engine-start-to-script time of 109.06 seconds and median engine-start-to-ready time of 336.65 seconds; the corresponding p95 values were 189.56 and 675.21 seconds. The ready timer begins inside the engine,

Worker cap	Cold wall (s)	Peak handlers	Warm wall (s)	Warm calls
1	6.001	1	0.054	0
4	1.690	4	0.020	0
16	0.777	16	0.082	0
32	0.626	30	0.136	0

Table 1: Synthetic sleep workload on one local machine (Darwin arm64, Python 3.14.7, SQLite 3.53.4); one cold and one warm run per width. No model, human review, figure parsing or media delivery is in these times. Source: `benchmarks/results/production-widths.json`.

so it excludes user queue wait. Parts share startup work and a media lane, and four have explicit model-call cache hits. These are correlated saved artifacts, not independent cold trials or a Lamina benchmark. In the same historical course batch, 19 completed *jobs* had median service time 575.54 seconds and median queue wait 5,042.11 seconds. A job can produce several parts. The separation of part execution from admission wait is exactly why Eq. 6 is a lower bound on service and cannot predict button-to-audio latency by itself. The measurements and archival procedure are summarized in [the workflow design account](#); they do not establish a speedup over another system.

The audio factory’s saved toxicology receipt illustrates the dependency choice more sharply: its director took 73.754 seconds, the slowest of eight section writers 4.122 seconds, and the slowest local check 2.318 seconds. These recorded call times include model-gate waiting. More writer slots could not remove the director from D . Moving a check to start when its own writer finishes could remove a stage-wide barrier, subject to provider capacity; this is a scheduling hypothesis rather than a measured improvement. For a PDF reference, family and global reconciliation likewise sit on D after local readers. For a staged case, the setter sits on D , while bounded key and prefix-solver work can overlap. The math becomes useful when it names these actual decisions and their competing resource demands.

A checked-in fixed-graph benchmark ran 24 sleep-only extraction jobs, two global steps and six author/review pairs. At worker limits 1, 4 and 8, cold times were 2.412, 0.865 and 0.595 seconds; warm cache times were about 0.018–0.022 seconds with zero handler calls. The integrated production path needs its own measurement with models and accepted artifacts.

A second SQLite/cache run used 64 sleep-only readers, two serial global jobs and 16 author/review chains, totaling 98 handlers. Table 1 gives one local run per width. At width 32, peak overlap was 30 because of serial stages and chain dependencies. Warm calls stayed at zero, though bookkeeping made their wall times non-monotone.

8 Cache identity, review, and repair

A cache key must include the actual request: source and policy versions, instructions, selected task fields, dependency results, adapter identity and selected observations. Unrelated changes can retain work; a changed source meaning or review policy must invalidate affected results. Cache hits record matching requests. Quality review examines the returned content and becomes stale when its artifact or source changes.

Findings should name the affected object, location, defect, source, severity, reviewer and artifact

revision. Examples include an unreadable region, missing answer, false merge, unsupported claim, wrong condition, broken cross-section reference, candidate-answer leak or delivery failure. The finding can reopen parsing, representation, selection, writing, rendering or publication. Conflicting source authority may require a human decision.

The transitive dependency closure is a candidate recheck set, not an exact repair bill. For observation o , define

$$R(o) = (s_o, m_o, h_o, d_o), \quad (7)$$

where s_o is re-executed service seconds, m_o reported model or external charges, h_o human review minutes and d_o replacement deliveries. Keep the units separate unless a conversion is declared. Production supports named section revisions and one bounded repair and re-review; the fixed guide can block export on review failure. Routing findings to upstream causes and independently judging the final correction remain limited.

9 Experience and method reuse

A cache reuses unchanged job output. Method learning would change a future choice of source policy, representation, window, graph, context, reviewer or format. Lamina stores receipts and operator observations. An operator can attach a same-family observation ID to a later method node or production plan, where it enters context. Selection is manual; a transfer test would measure whether that choice improves the next output.

Treat a method revision as a testable proposal: name the failure, changed decision, preserved behavior and added cost. Compare old and new versions on sources outside tuning, including a task where the favored approach should fail. Record losses. DSPy already studies metric-driven optimization of LLM pipelines [4]; Lamina’s question is whether task-specific source and repair decisions improve accepted documents under change.

10 Worked example: a disputed timeout

A fictional team is writing an export-worker brief. Its current specification sets a 90-second lease, renewal every 30 seconds while healthy, cessation of output after failed renewal, and reassignment only after expiry. A historical runbook expected 60 seconds and describes an incident where a second worker began while the first appeared to write. The team checks the incident’s configured timeout and cleanup behavior before changing its retry timer.

Independent readers preserve both source claims with excerpts and conditions. Reconciliation applies the owner’s authority policy: the 60-second rule stays historical, and the incident’s cause stays unresolved. Behavior and operations writers use one evidence board in parallel. A reviewer sees both drafts and the board to find lost qualifiers or unsupported recommendations. The portable six-node technical-brief example validates this graph and context wiring; no model-written brief from it has been independently accepted.

If the current lease changes, its reader and the board must rerun. Both writers may need new context even when only one visibly states the number. The historical reader can remain cached when its source and task inputs match. A cleanup finding points to a source gap, so the brief should say to

check the actual cleanup contract. A repair receipt should identify source version, reopened decisions, executed and reused jobs, review and delivered file. The portable example exercises the graph and context handoff; the full correction journey remains an evaluation task.

A separate deterministic production fixture uses two original short sources and three sections. Its first run flags and repairs one section, with no remaining finding. Revising section three hits cache on six of eight requests; its write and review rerun, while the first two bodies remain byte-identical. Canned JSON responses test routing and reuse, not model judgment. Their millisecond timings are fixture timings.

11 Current implementation

The fixed guide builder ingests Markdown, text and extractable PDF text with hashes, units and locators. It reads with adjacent context, reconciles concepts, plans, writes, reviews and exports source-linked material. Validation checks ownership, quotations, membership and output shape.

Capability	Present mechanism	Status boundary
Source identity and policy	Hashes, ordered units, locators, authority/supplement/historical/form roles	Working for supported text; model’s use of authority still needs review
Fixed guide	Extract, reconcile, plan, author, review and export	Working application; one teaching route
Reusable method	User-declared JSON jobs, dependencies, lanes, cache and receipts	Working separately; semantic output rules external
Source-aware production	Bounded reading, shared route, section production and revision	Working local engine/API/CLI; semantic quality unproven
Retrieval targets	Opt-in guide/assessment catalog partitions extracted ideas into source-backed prompts and answer groups	Structural ownership checked; semantic equivalence unproven
Assessment check	Every assessment runs candidate-only solve, then separate judge with key and exact excerpts	Working local stage; no interactive administration or provider-memory guarantee
Audio delivery	Optional command adapter, PCM WAV check, hashes and local render lock	Working local file verification; no listened or ASR quality check
Experience	Operator-selected observations enter named future nodes	Explicit reuse; no automatic adaptation
Local delivery and recovery	HTML, Markdown, optional PDF, report, plan and project resume	Working locally; external publication separate
Typed upstream repair	Finding-to-cause routing across source, representation and output	Design target beyond section repair

Table 2: Status distinctions in the current checkout. Local end-to-end wiring and deterministic tests establish mechanics; they do not establish human-accepted model quality.

The separate method runtime runs user-declared JSON jobs with hard dependencies, selected task fields, capacity lanes, cached receipts and manually selected observations. It checks for JSON objects;

declared shapes remain hints. Production reads factual teaching sources in owned windows, passes form exemplars separately, lets a model choose sections and representations, and writes and reviews under separate capacities. The optional guide/assessment retrieval stage gives each extracted idea one target and source-exact answer items. Assigned targets reach the guide or examiner artifact; omissions stay in the plan and report.

Production repairs a flagged section once and rechecks it. Named revisions reuse unchanged requests. The local project API persists state, marks unfinished projects interrupted after restart and resumes from plans and cache. CLI and Projects share the engine. Export produces Markdown, HTML, plan, report and optional PDF. Assessment keeps candidate and examiner files separate; every assessment runs candidate-only answers followed by key-and-evidence judging. A configured audio adapter renders ready podcast scripts to PCM WAV, checks file structure, duration and hashes, and records synthesis input. A local per-user file lock serializes uncached adapter calls.

12 Limits

Text extraction cannot recover arbitrary figures or prove that a citation entails a claim. Target ownership cannot validate semantic equivalence or material the reader missed. Separate candidate and judge calls do not erase a provider’s hidden state or administer a live case. PCM validation does not check spoken words, pronunciation or listener use. Production does not infer every cross-section dependency or route a missing idea through upstream parsing and planning. The method runtime remains a separate path without automatic method selection or demonstrated transfer. The current fixtures establish mechanics; judged output quality and comparative advantage require the study below.

13 Evaluation and competitive position

Compare time and total cost to a *quality-matched accepted artifact*, including operator work. Freeze a new permissioned corpus and task contract. Include a difficult table or figure, overlap, a contradiction and an omission risk. Independent reviewers inventory important source objects, conditions and acceptable omissions from rendered originals, keeping disagreements. Hold evaluation questions out of generation. Compare a careful source-order workflow and, where feasible, fully configured external products under the same task rules. Hold the model/provider fixed for an internal runtime comparison; across products, report the complete system result.

Record first useful draft and accepted accessible artifact times; source misses, false merges and unsupported claims; editorial usability; tokens, observed provider charges, human minutes, retries, failed attempts and delivery checks. Label estimated API-equivalent costs separately. Rotate paired run order and report median, tails, range and sample size. Then change a source, seed local and global findings, and ask a fresh operator to repair and deliver. Check whether the reduced repair set was safe. A fresh-agent run tests transfer without the original author.

Manus documents wide research and evolving project context [8, 9]. NotebookLM generates source-based audio [3]. LangGraph and Google’s ADK support graph workflows, including flexible and dynamic workflows in ADK’s current documentation [2, 6]. The possible Lamina advantage is nar-

rower: an operator may be able to explain and repair a source-to-artifact decision with less lost work at matched quality. This remains unmeasured.

14 Conclusion

Lamina gives code responsibility for identity, source boundaries, scheduling, persistence and receipts while models read, plan and write. Its guide, method and production paths cover different parts of that work. The next test is a new-source artifact with a real correction, recorded costs and human judgment of the delivered result.

References

- [1] Robert D. Blumofe and Charles E. Leiserson. Scheduling multithreaded computations by work stealing. *Journal of the ACM*, 46(5):720–748, 1999. doi: 10.1145/324133.324234. URL <https://doi.org/10.1145/324133.324234>.
- [2] Google. Agent development kit: Workflows, 2026. URL <https://github.com/google/adk-docs/blob/main/docs/workflows/index.md>. Official ADK documentation, accessed 21 September 2026.
- [3] Google. Generate an audio overview in notebooklm, 2026. URL <https://support.google.com/gemininotebook/answer/16212820?hl=en>. Official product help, accessed 21 September 2026.
- [4] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html.
- [5] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An llm compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 24370–24391, 2024. URL <https://proceedings.mlr.press/v235/kim24y.html>.
- [6] LangChain. Langgraph graph api, persistence, and interrupts, 2026. URL <https://docs.langchain.com/oss/python/langgraph/graph-api>. Official documentation, accessed 21 September 2026.
- [7] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl_a_00638. URL <https://aclanthology.org/2024.tacl-1.9/>.
- [8] Manus. What is wide research?, 2026. URL <https://help.manus.im/en/articles/11960169-what-is-wide-research>. Official product documentation, accessed 21 September 2026.
- [9] Manus. Projects that learn from every task, 2026. URL <https://manus.im/blog/manus-projects-self-updating>. Official product description, accessed 21 September 2026.